

Mimetic Neural Networks (MimNet)

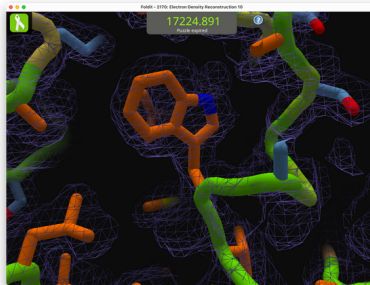
Konrad Ciborowski, Szymon Szulc

Inspiracja

Structure Solving

There are millions of different proteins in the natural world, and each one folds into a unique structure. **Solving a protein's structure is essential to understand how the protein functions**, and can lead researchers to new paths for treating disease. Since proteins are too small to see under a microscope, scientists use indirect methods to get clues about how each protein folds. We use those clues to solve the protein's structure.

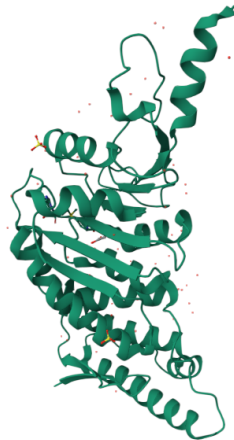
Foldit is especially useful for solving protein structures with electron density maps. These maps show the overall shape of a folded protein, although sometimes maps are noisy or difficult to interpret. Try out puzzles in the [Electron Density Category](#) to help scientists solve the structure of natural proteins.



Wprowadzenie

- Zwijanie białek: przewidywanie struktury 3D z sekwencji aminokwasów.
- Projektowanie białek: odwrotność zwijania – znajdowanie sekwencji dla danej struktury.
- Struktura białka definiuje jego funkcje.
- MimNet: jedna sieć neuronowa do obu zadań, inspirowana dynamiką fizyczną.

Struktura białka



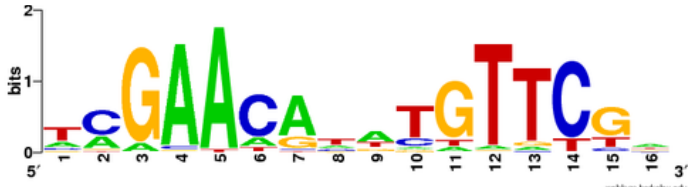
Główne idee

- Sieci mimetyczne: naśladują dynamikę Hamiltona.
- Odwracalna architektura umożliwia propagację w obu kierunkach.
- Wykorzystanie grafowych sieci konwolucyjnych (GCN) i U-Net.

Architektura MimNet

- Wejście: macierz $20 \times n$ one-hot aminokwasów + macierz PSSM (z grubsza prawdopodobieństwo wystąpienia aminokwasu na danej pozycji)
- Wyjście: koordynaty 3D każdego aminokwasu

PSSM



Architektura MimNet

Model propagacji (Verlet):

$$Y_0 = \text{Conv1D}([S, S^+]), \quad n_f \text{ kanałów}$$

$$Y_{j+1} = 2Y_j - Y_{j-1} + h^2 f(Y_j)$$

$$\hat{X} = \text{Conv1D}(Y_T), \quad 3 \text{ kanały}$$

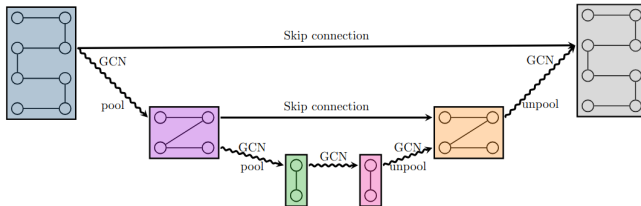
- Schemat nie traci informacji.

Splot grafowy (GCN) – nasze f

$$f(Y_j) = -L^* \times \text{ReLU}(L \times Y_j)$$

gdzie L to wymyślny Laplasjan

U-Net



Optymalizator Adam

Adam (Adaptive Moment Estimation) to popularny optymalizator w deep learningu, łączący zalety AdaGrad i RMSProp.

Aktualizacja wag:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Funkcje strat

- Projektowanie: KL-divergencja między PSSM a przewidywaną sekwencją.
- Zwijanie: dRMSD – macierz odległości.

Krzywa uczenia

